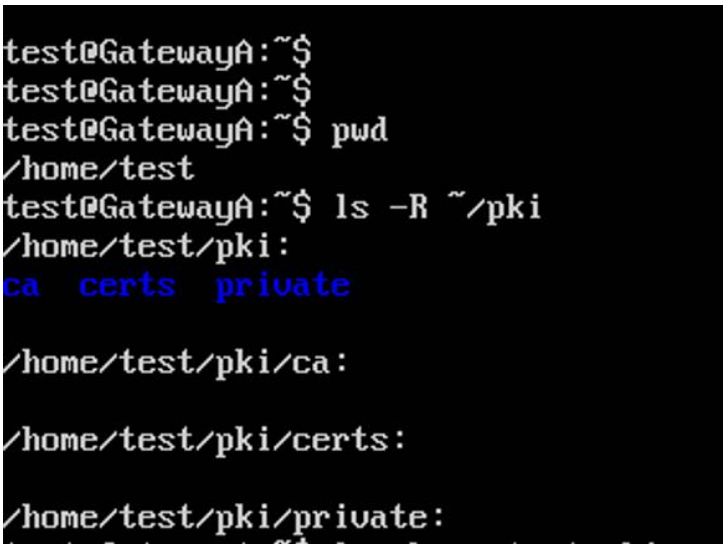
IPSec VPN Lab — Gateway-to-Gateway

Phase 5

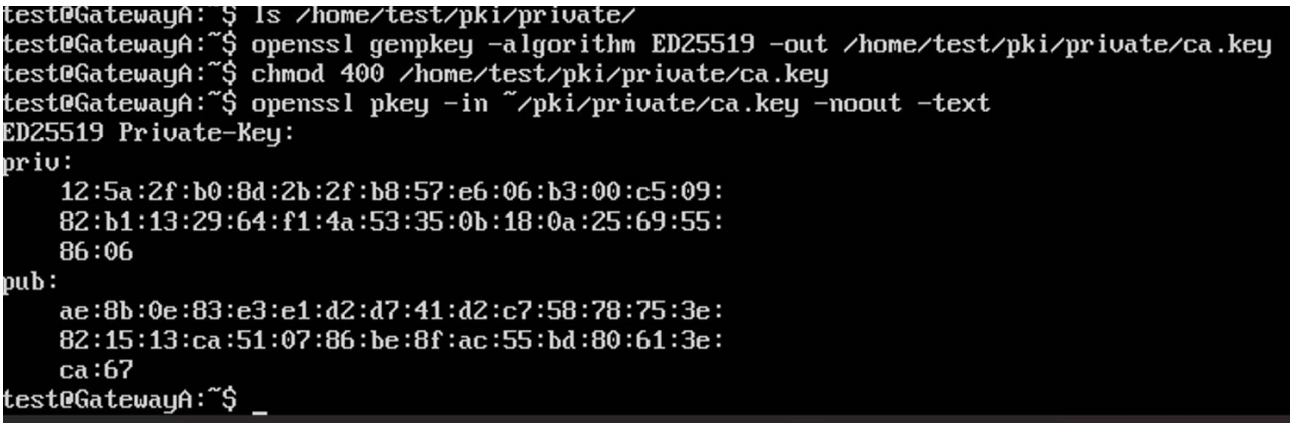
5.1 Create PKI Directory Structure (VM A)



What we did: We created a dedicated, secured folder hierarchy (ca, certs, private) on Gateway A.

Why it's needed: This establishes an organized Security Workspace. By using **chmod 700** on the private folder, we ensure that the most sensitive files (private keys) are physically inaccessible to any unauthorized users on the system.

Step 5.2 — Generate the Root CA Private Key (Ed25519)



What we did: We generated an Ed25519 private key to act as the master key for our Certificate Authority.

Why it's needed: This key allows Gateway A to sign and "stamp" the identities of every other machine in the network.

Step 5.3 — Create the Self-Signed CA Certificate

```
test@GatewayA:~$ openssl req -x509 -new -nodes -key ~/pki/private/ca.key -sha256 -days 3650 -out ~/pki/ca/ca.crt -subj "/CN=IPSec-Lab-CA/O=Lab/C=PL"
test@GatewayA:~$ openssl x509 -in ~/pki/ca/ca.crt -noout -text | grep -E "Issuer|CA:|Public Key Algorithm"
        Issuer: CN = IPSec-Lab-CA, O = Lab, C = PL
            Public Key Algorithm: ED25519
                CA:TRUE
test@GatewayA:~$
```

What we did: We transformed the private key into a public CA Certificate.

Why it's needed: We established the Ultimate Source of Trust on VM A. This certificate is the "Root of Trust"; every other gateway will trust each other only because they both recognize this specific CA certificate as their main.

Step 5.4 — Generate Gateway A Certificate (Ed25519)

```
test@GatewayA:~$ openssl x509 -req -in ~/pki/certs/gw-a.csr -CA ~/pki/ca/ca.crt -CAkey ~/pki/private/ca.key -CAcreateserial -out ~/pki/certs/gw-a.crt -days 365
-extfile /tmp/gw-a-ext.cnf -extensions v3_req
Certificate request self-signature ok
subject=CN = 192.168.100.1, O = Lab, C = PL
```

Verify SAN and key type:

```
test@GatewayA:~$ openssl x509 -in ~/pki/certs/gw-a.crt -noout -text | grep -E "Subject Alternative|Public Key Algorithm"
            Public Key Algorithm: ED25519
            X509v3 Subject Alternative Name:
test@GatewayA:~$ openssl x509 -in ~/pki/certs/gw-a.crt -noout -text | grep -A 1 "Subject Alternative" -B 1 "Public Key Algorithm"
grep: Public Key Algorithm: No such file or directory
test@GatewayA:~$ openssl x509 -in ~/pki/certs/gw-a.crt -noout -text | grep -A 2 "Subject Alternative Name"
            X509v3 Subject Alternative Name:
                IP Address:192.168.100.1
            X509v3 Extended Key Usage:
test@GatewayA:~$ _
```

Step 5.5 — Generate Gateway B Certificate (Ed25519)

```
test@GatewayA:~$
test@GatewayA:~$ openssl genpkey -algorithm ED25519 -out ~/pki/private/gw-b.key
test@GatewayA:~$ chmod 400 ~/pki/private/gw-b.key
test@GatewayA:~$ openssl req -new \
> -key ~/pki/private/gw-b.key \
> -out ~/pki/certs/gw-b.csr \
> -subj "/CN=192.168.100.2/O=Lab/C=PL"
test@GatewayA:~$ cat > /tmp/gw-b-ext.cnf << 'EOF'
> [v3_req]
> subjectAltName = IP:192.168.100.2
> extendedKeyUsage = serverAuth, clientAuth
> keyUsage = DigitalSignature
> EOF
test@GatewayA:~$ openssl x509 -req \
> -in ~/pki/certs/gw-b.csr \
> -CA ~/pki/ca/ca.crt \
> -CAkey ~/pki/private/ca.key \
> -CAcreateserial \
> -out ~/pki/certs/gw-b.crt \
> -days 365 \
> -extfile /tmp/gw-b-ext.cnf \
> -extensions v3_req
Error checking extension section v3_req
4057C445527E0000:error:1100006F:X509 V3 routines:v2i_ASN1_BIT_STRING:unknown bit string argument:../crypto/x509/v3_bitst.c:84:DigitalSignature
4057C445527E0000:error:11000080:X509 V3 routines:X509V3_EXT_nconf_int:error in extension:../crypto/x509/v3_conf.c:48:section=v3_req, name=keyUsage, value=Digit
lSignature
test@GatewayA:~$ cat > /tmp/gw-b-ext.cnf << 'EOF'
[v3_req]
subjectAltName = IP:192.168.100.2
extendedKeyUsage = serverAuth, clientAuth
keyUsage = digitalSignature
EOF
test@GatewayA:~$ openssl x509 -req -in ~/pki/certs/gw-b.csr -CA ~/pki/ca/ca.crt -CAkey ~/pki/private/ca.key -CAcreateserial -out ~/pki/certs/gw-b.crt -days 365
-extfile /tmp/gw-b-ext.cnf -extensions v3_req
Certificate request self-signature ok
subject=CN = 192.168.100.2, O = Lab, C = PL
test@GatewayA:~$
```

Verify both certs are signed by the same CA:

```
test@GatewayA:~$ openssl verify -CAfile ~/pki/ca/ca.crt ~/pki/certs/gw-a.crt
/home/test/pki/certs/gw-a.crt: OK
test@GatewayA:~$ openssl verify -CAfile ~/pki/ca/ca.crt ~/pki/certs/gw-b.crt
/home/test/pki/certs/gw-b.crt: OK
test@GatewayA:~$
```

Step 5.4 & 5.5:

What we did: We created specific certificates for Gateway A and Gateway B, including Subject Alternative Name (SAN) extensions.

Why it's needed: This provides each gateway with a "Digital Passport." The SAN (the IP address) is critical because modern IKEv2 protocols (like strongSwan) will reject a connection if the certificate doesn't explicitly match the IP address of the sender.

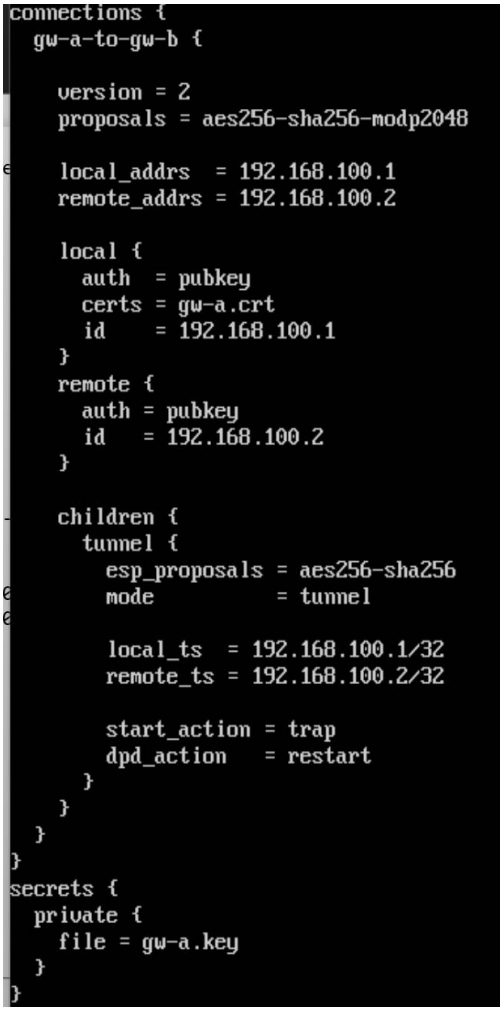
Step 5.6 — Copy Gateway B Files to VM B

```
test@GatewayA:~$ scp ~/pki/ca/ca.crt test@192.168.100.2:/tmp/ca.crt
test@192.168.100.2's password:
ca.crt                                        100%  570   104.7KB/s   00:00
test@GatewayA:~$ scp ~/pki/certs/gw-b.crt test@192.168.100.2:/tmp/gw-b.crt
test@192.168.100.2's password:
gw-b.crt                                      100%  631    65.3KB/s   00:00
test@GatewayA:~$ scp ~/pki/private/gw-b.key test@192.168.100.2:/tmp/gw-b.key
test@192.168.100.2's password:
gw-b.key                                      100%  119    35.4KB/s   00:00
test@GatewayA:~$ _
```

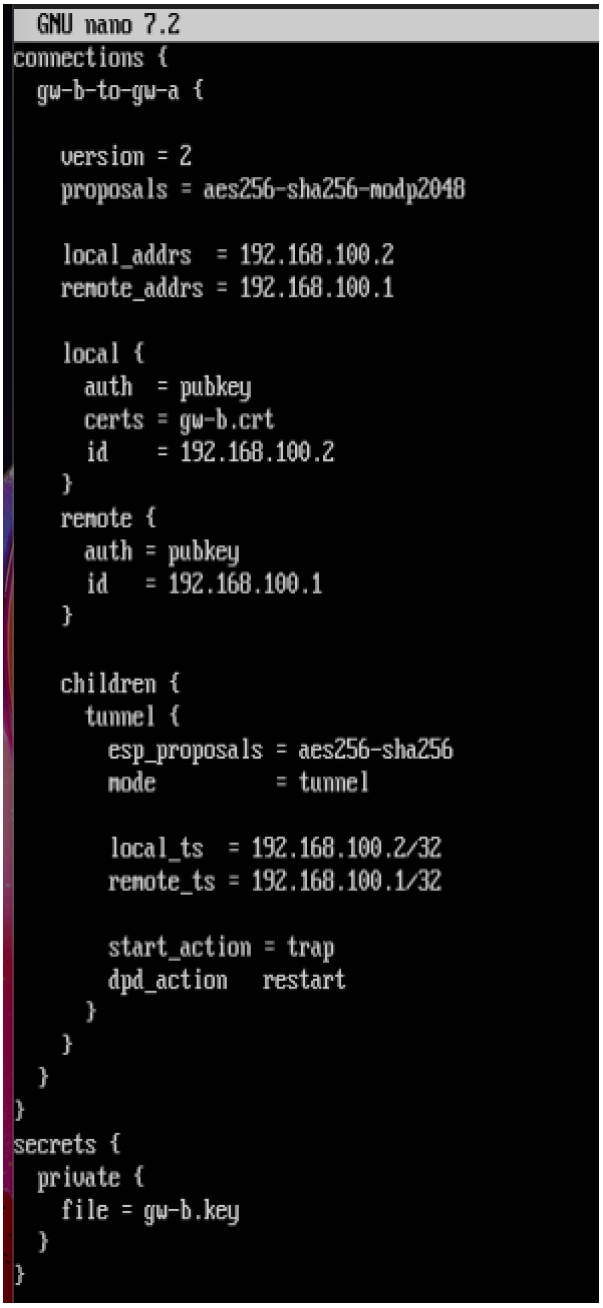
What we did: We used scp to securely move the CA certificate, Gateway B's certificate, and Gateway B's private key to the second VM.

Why it's needed: This achieves Secure Distribution. Gateway B now has its own unique identity and the "template" (the CA cert) it needs to verify Gateway A's identity later.

Step 6.3 — Configure VM A (`swanctl.conf`)



Step 6.4 — Configure VM B (`swanctl.conf`)



Step 6.3 & 6.4:

What we did: We wrote the "blueprint" for the tunnel, defining local/remote addresses and encryption proposals.

Why it's needed: We established the **Peer Authorization Database (PAD)**. This is needed to tell the software exactly which encryption to use and which IPs are allowed to enter the tunnel.

Phase 7 — Establish the IPSec Tunnel

Step 7.1 — Start strongSwan (Both VMs)

```
test@GatewayA:~$ sudo swanctl --load-all
loaded certificate from '/etc/swanctl/x509/gw-a.crt'
loaded certificate from '/etc/swanctl/x509ca/ca.crt'
loaded ED25519 key from '/etc/swanctl/private/gw-a.key'
no authorities found, 0 unloaded
no pools found, 0 unloaded
loaded connection 'gw-a-to-gw-b'
successfully loaded 1 connections, 0 unloaded
test@GatewayA:~$
```

What we did: We used systemctl to enable and start the strongSwan daemon.

Why it's needed: This wakes up the daemon, which is the engine that handles all the IKEv2 logic and communicates with the Linux Kernel.

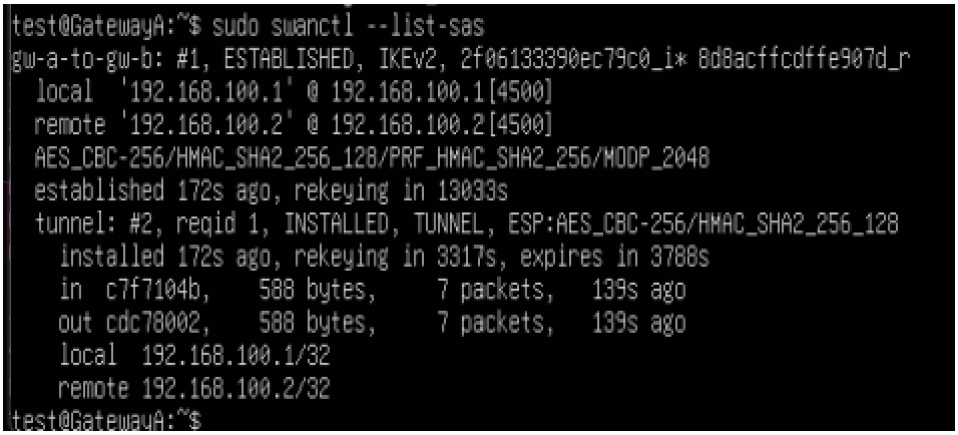
Step 7.2 — Load Configuration (Both VMs)

```
test@GatewayB:~$ sync
test@GatewayB:~$ sudo swanctl --load-all
loaded certificate from '/etc/swanctl/x509/gw-b.crt'
loaded certificate from '/etc/swanctl/x509ca/ca.crt'
loaded ED25519 key from '/etc/swanctl/private/gw-b.key'
no authorities found, 0 unloaded
no pools found, 0 unloaded
loaded connection 'gw-b-to-gw-a'
successfully loaded 1 connections, 0 unloaded
test@GatewayB:~$
```

Step 7.3 — Initiate the Tunnel (from VM A)

```
test@GatewayA:~$ sudo swanctl --initiate --child tunnel
[sudo] password for test:
[IKE] initiating IKE_SA gw-a-to-gw-b[1] to 192.168.100.2
[ENC] generating IKE_SA_INIT request 0 [ SA KE No N(NATD_S_IP) N(NATD_D_IP) N(FRAG_SUP) N(HASH_ALG) N(REDIR_SUP) ]
[NET] sending packet: from 192.168.100.1 to 192.168.100.2[500] (464 bytes)
[IKE] retransmit 1 of request with message ID 0
[NET] sending packet: from 192.168.100.1 to 192.168.100.2[500] (464 bytes)
[IKE] retransmit 2 of request with message ID 0
[NET] sending packet: from 192.168.100.1 to 192.168.100.2[500] (464 bytes)
```

Step 7.4 — Verify Tunnel Status

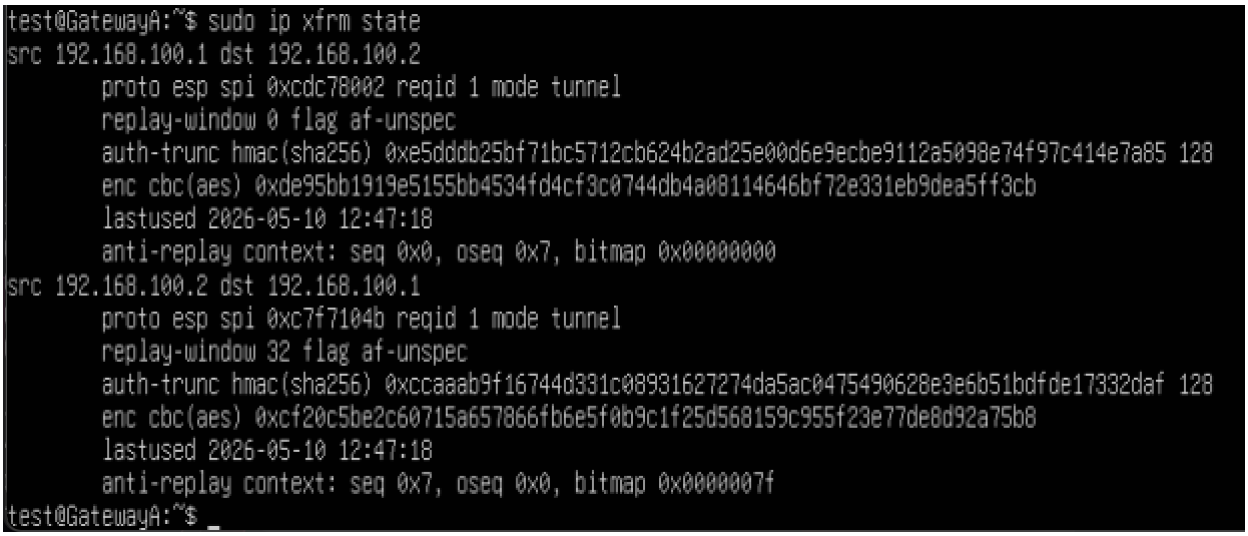


Step 7.3 & 7.4:

What we did: We triggered the handshake and ran sudo swanctl --list-sas.

Why it's needed: We achieved a **Live Security Association**. This proves the gateways successfully verified each other's certificates and established a secure channel.

Step 7.5 — Inspect the SAD (Security Association Database)



Step 7.6 — Inspect the SPD (Security Policy Database)

```
test@GatewayA:~$ sudo ip xfrm policy
src 192.168.100.1/32 dst 192.168.100.2/32
        dir out priority 367231
        tmpl src 192.168.100.1 dst 192.168.100.2
                proto esp spi 0xcdc78002 reqid 1 mode tunn
src 192.168.100.2/32 dst 192.168.100.1/32
        dir fwd priority 367231
        tmpl src 192.168.100.2 dst 192.168.100.1
                proto esp reqid 1 mode tunnel
src 192.168.100.2/32 dst 192.168.100.1/32
        dir in priority 367231
        tmpl src 192.168.100.2 dst 192.168.100.1
                proto esp reqid 1 mode tunnel
src 0.0.0.0/0 dst 0.0.0.0/0
        socket in priority 0
src 0.0.0.0/0 dst 0.0.0.0/0
```

Step 7.5 & 7.6: Inspect the SAD and SPD

Why it's needed: We verified the **Kernel Integration**. This proves that the strongSwan software (User Space) successfully programmed the Linux Kernel (Kernel Space) to handle the actual encryption of packets.

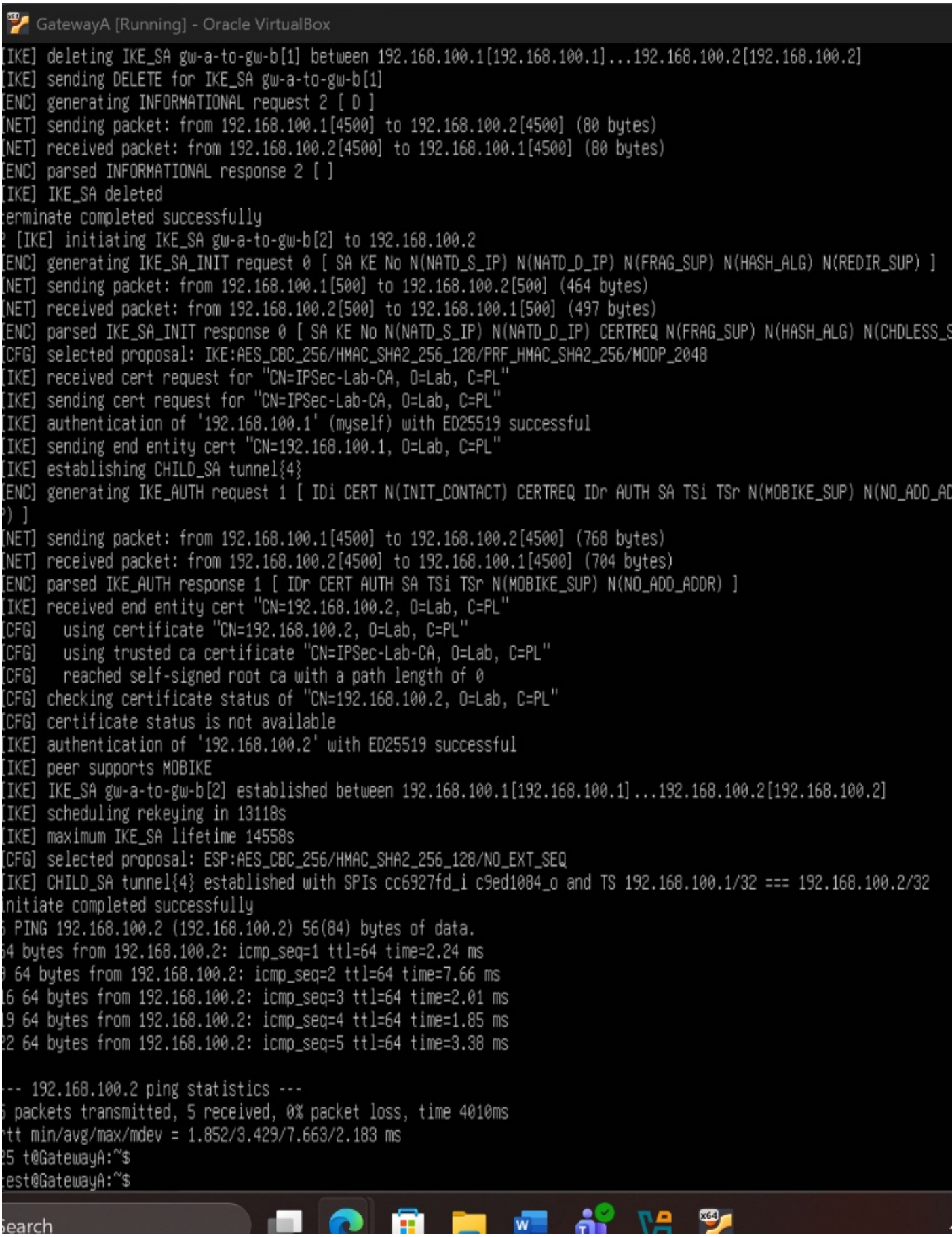
Step 7.7 — Send Traffic Through the Tunnel

```
est@GatewayA:~$ ping -c 10 192.168.100.2
PING 192.168.100.2 (192.168.100.2) 56(84) bytes of data.
64 bytes from 192.168.100.2: icmp_seq=1 ttl=64 time=1.63 ms
64 bytes from 192.168.100.2: icmp_seq=2 ttl=64 time=0.951 ms
64 bytes from 192.168.100.2: icmp_seq=3 ttl=64 time=3.57 ms
64 bytes from 192.168.100.2: icmp_seq=4 ttl=64 time=2.35 ms
64 bytes from 192.168.100.2: icmp_seq=5 ttl=64 time=1.97 ms
64 bytes from 192.168.100.2: icmp_seq=6 ttl=64 time=2.89 ms
64 bytes from 192.168.100.2: icmp_seq=7 ttl=64 time=5.04 ms
64 bytes from 192.168.100.2: icmp_seq=8 ttl=64 time=2.30 ms
64 bytes from 192.168.100.2: icmp_seq=9 ttl=64 time=2.74 ms
64 bytes from 192.168.100.2: icmp_seq=10 ttl=64 time=1.42 ms

-- 192.168.100.2 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9026ms
tt min/avg/max/mdev = 0.951/2.485/5.041/1.116 ms
est@GatewayA:~$
```

Phase 8 — Traffic Capture & Analysis

Step 8.2 — Capture the Full IKE Handshake



Step 8.3 — Quick CLI Analysis with tshark

Show IKE traffic only:

```
tshark: You don't have permission to read the file "/tmp/ipsec_full.pcap".
test@GatewayA:~$ sudo tshark -r /tmp/ipsec_full.pcap -Y 'udp.port == 500' -T fields -e frame.number -e ip.src -e ip.dst -e isakmp.exchangetype
Running as user "root" and group "root". This could be dangerous.
3       192.168.100.1   192.168.100.2   34
4       192.168.100.2   192.168.100.1   34
test@GatewayA:~$ _
```

Show ESP packets only:

```
test@GatewayA:~$ sudo tshark -r /tmp/ipsec_full.pcap -Y 'esp' -T fields -e frame.number -e ip.src -e ip.dst -e esp.spi -e esp.sequenc
Running as user "root" and group "root". This could be dangerous.
7       192.168.100.1   192.168.100.2   0xc9ed1084      1
8       192.168.100.2   192.168.100.1   0xcc6927fd      1
10      192.168.100.1   192.168.100.2   0xc9ed1084      2
11      192.168.100.2   192.168.100.1   0xcc6927fd      2
17      192.168.100.1   192.168.100.2   0xc9ed1084      3
18      192.168.100.2   192.168.100.1   0xcc6927fd      3
20      192.168.100.1   192.168.100.2   0xc9ed1084      4
21      192.168.100.2   192.168.100.1   0xcc6927fd      4
23      192.168.100.1   192.168.100.2   0xc9ed1084      5
24      192.168.100.2   192.168.100.1   0xcc6927fd      5
test@GatewayA:~$
```

Protocol hierarchy summary:

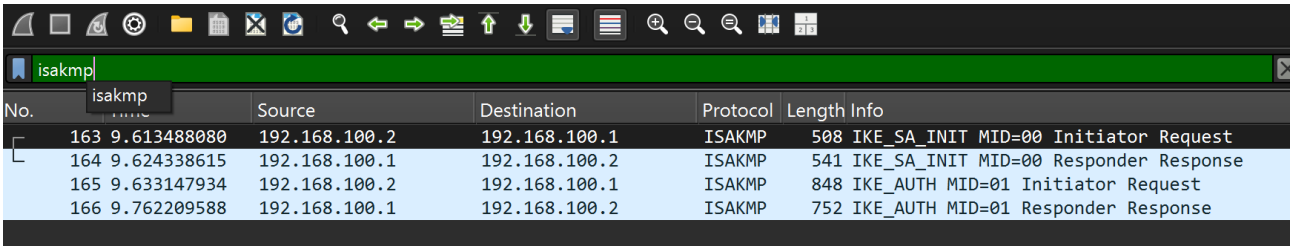
```
test@GatewayA:~$ sudo tshark -r /tmp/ipsec_full.pcap -q -z io,phs
Running as user "root" and group "root". This could be dangerous.

===================================================================
Protocol Hierarchy Statistics
Filter:

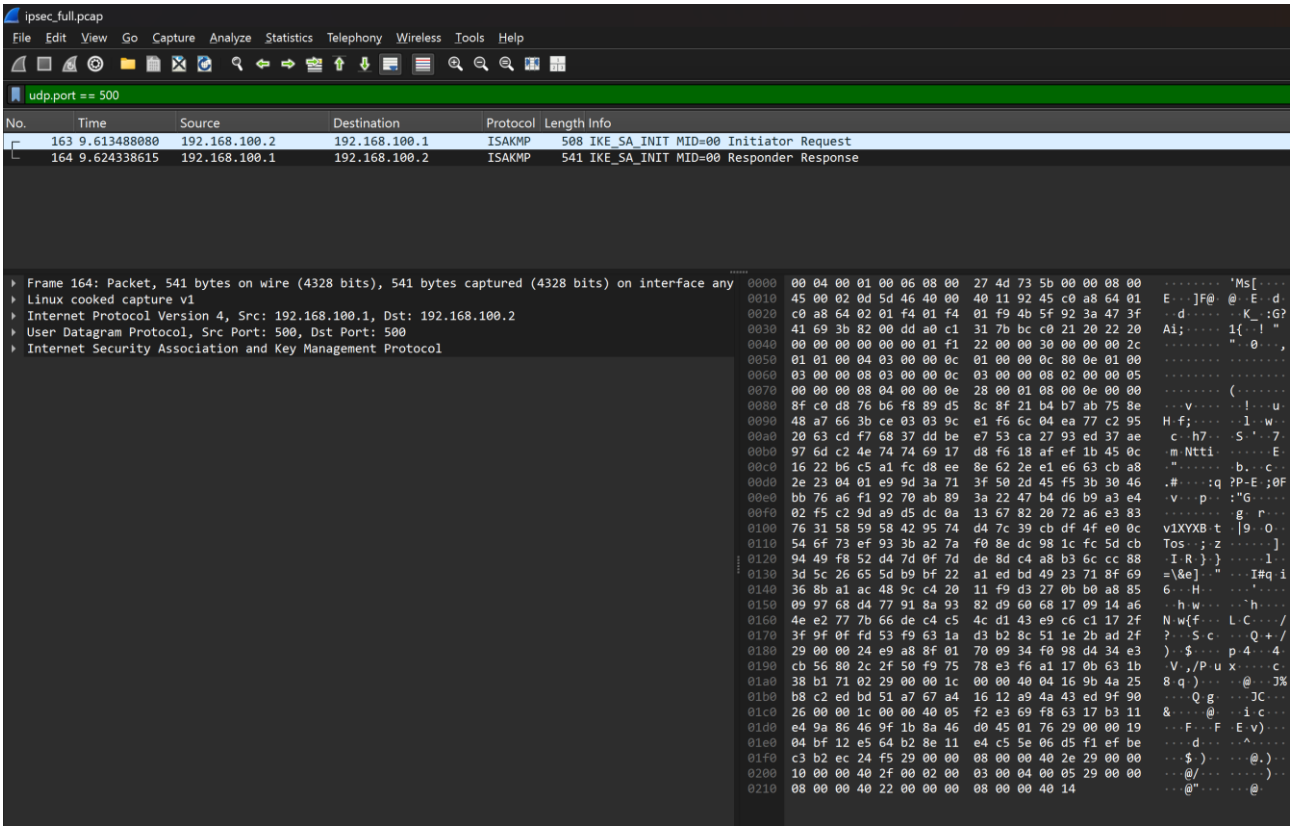
eth                                      frames:25 bytes:5255
  ip                                     frames:21 bytes:5051
    udp                                  frames:6 bytes:2861
      udpencap                           frames:4 bytes:1816
        isakmp                           frames:4 bytes:1816
      isakmp                             frames:2 bytes:1045
    esp                                  frames:10 bytes:1700
    icmp                                 frames:5 bytes:490
  arp                                    frames:4 bytes:204
===================================================================
test@GatewayA:~$
```

Step 8.5 — Wireshark Display Filters

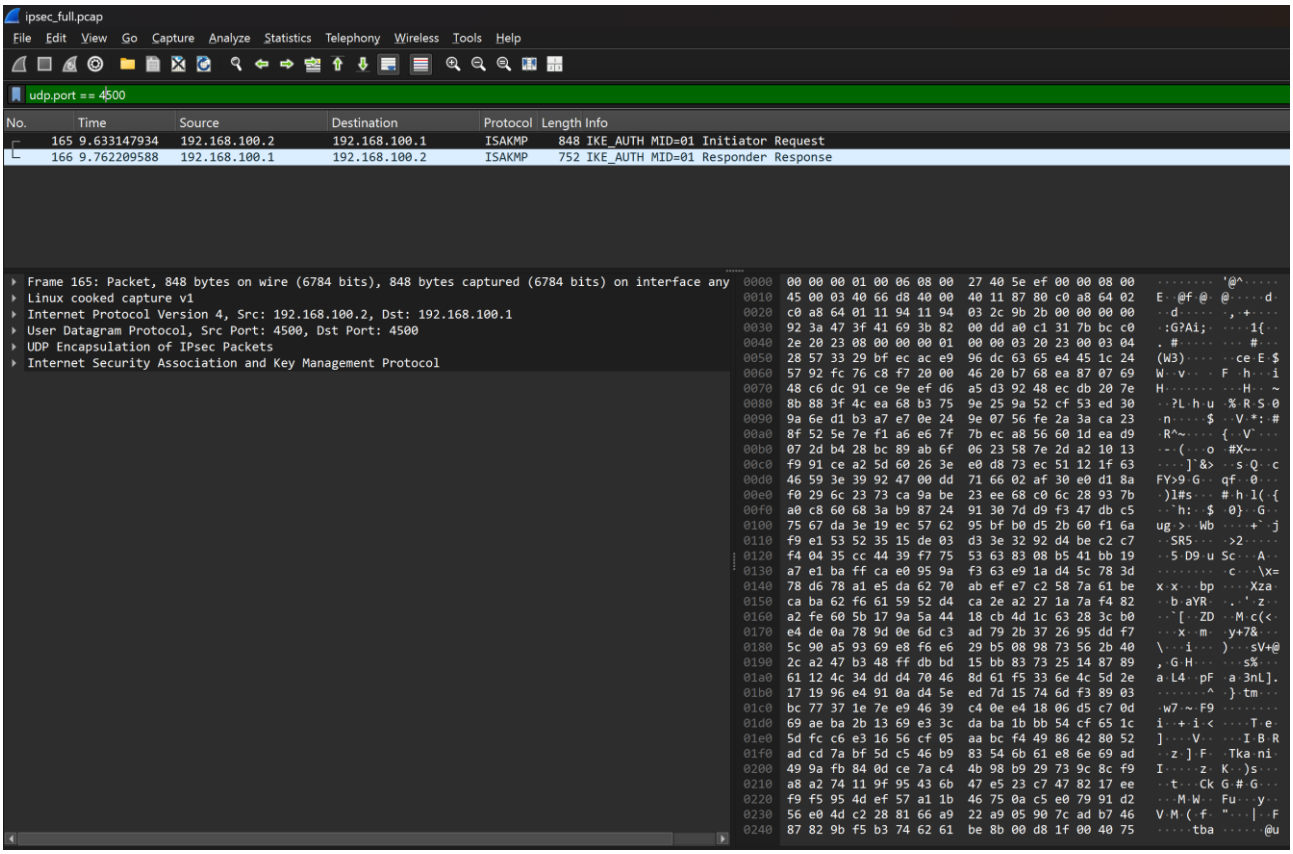
Isakmp:



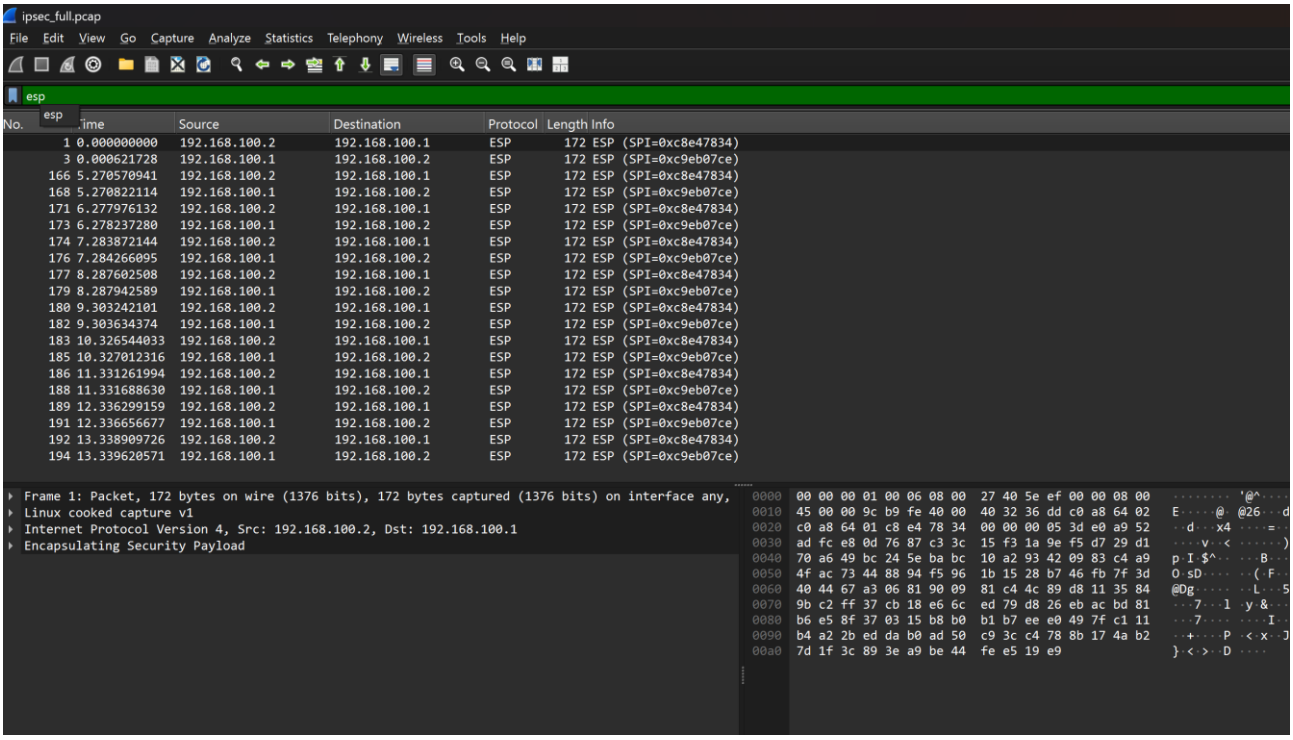
udp.port == 500:



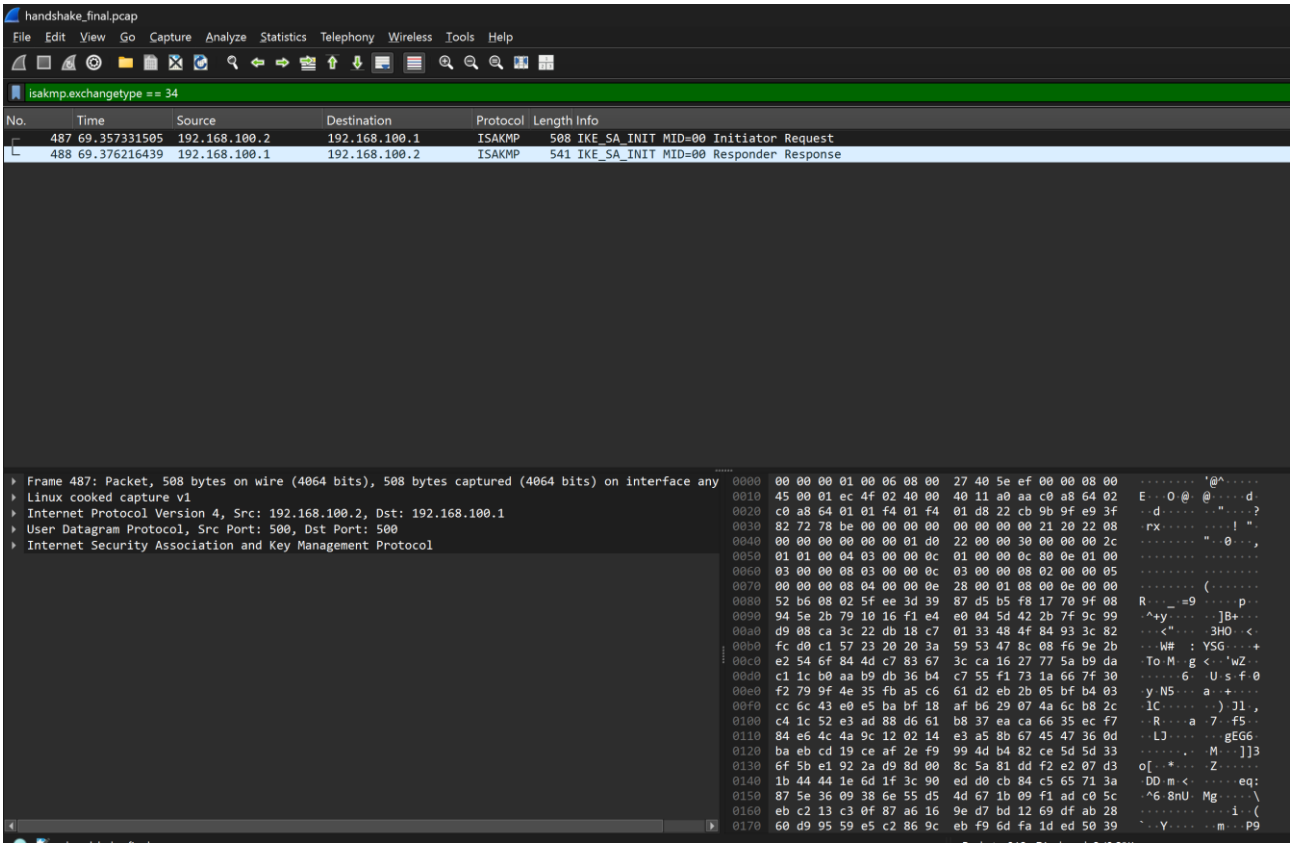
udp.port == 4500:



Esp:

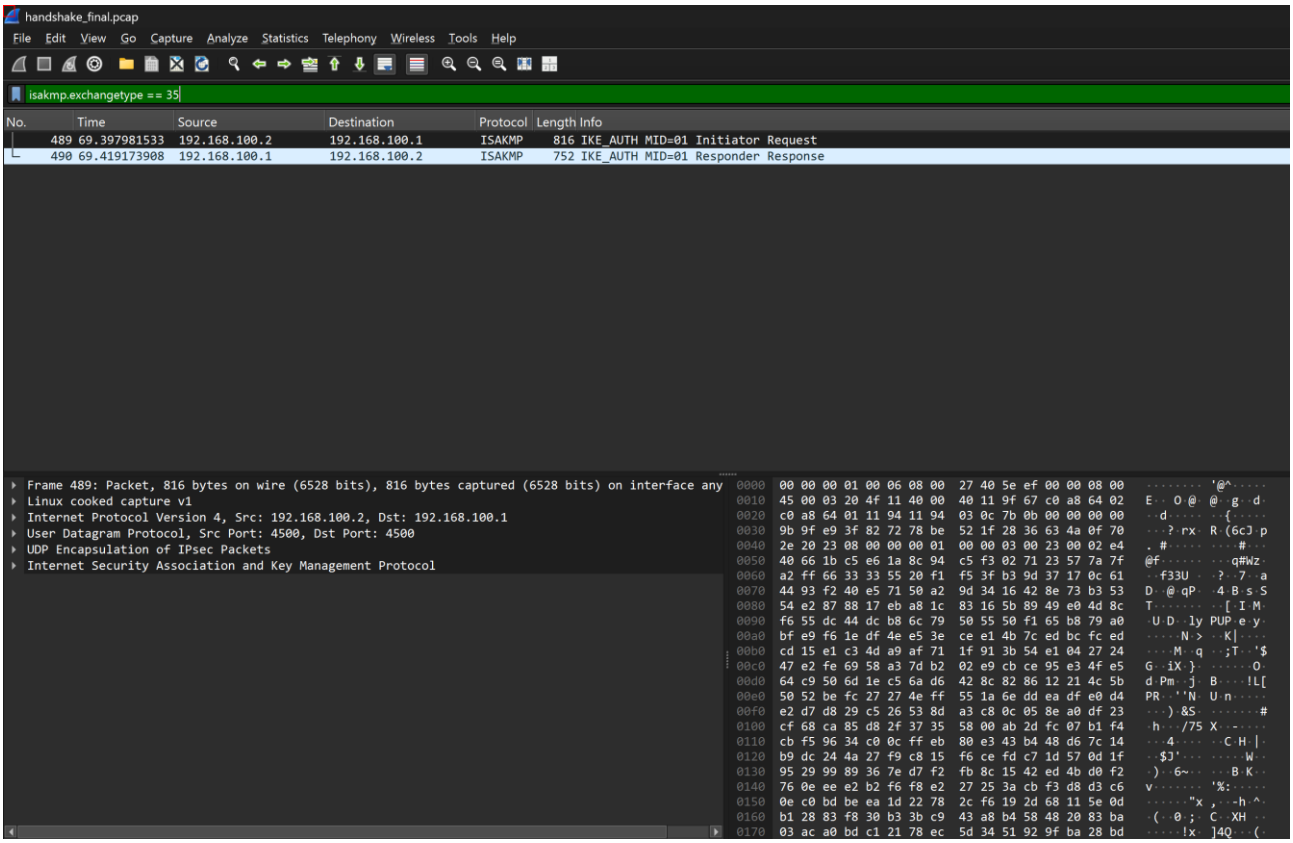


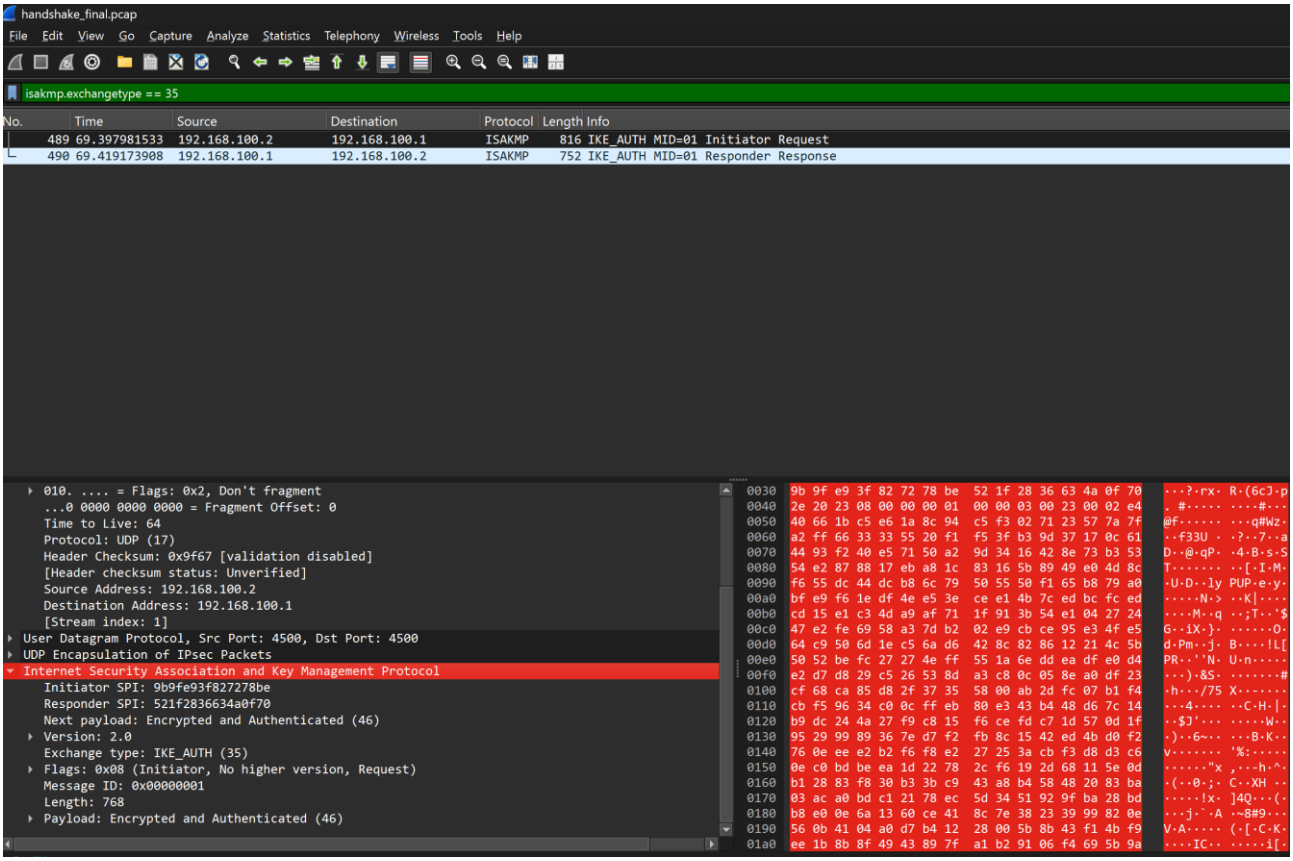
isakmp.exchangetype == 34:



```
Internet Security Association and Key Management Protocol
    Initiator SPI: 9b9fe93f827278be
    Responder SPI: 0000000000000000
    Next payload: Security Association (33)
  ▸ Version: 2.0
    Exchange type: IKE_SA_INIT (34)
  ▸ Flags: 0x08 (Initiator, No higher version, Request)
    Message ID: 0x00000000
    Length: 464
  ▸ Payload: Security Association (33)
  ▸ Payload: Key Exchange (34)
  ▸ Payload: Nonce (40)
  ▸ Payload: Notify (41) - NAT_DETECTION_SOURCE_IP
  ▸ Payload: Notify (41) - NAT_DETECTION_DESTINATION_IP
  ▸ Payload: Notify (41) - IKEV2_FRAGMENTATION_SUPPORTED
  ▸ Payload: Notify (41) - SIGNATURE_HASH_ALGORITHMS
  ▸ Payload: Notify (41) - REDIRECT_SUPPORTED
```

isakmp.exchangetype == 35:




handshake_final.pcap
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
isakmp.exchangetype == 35
No. Time Source Destination Protocol Length Info
489 69.397981533 192.168.100.2 192.168.100.1 ISAKMP 816 IKE_AUTH MID=01 Initiator Request
490 69.419173908 192.168.100.1 192.168.100.2 ISAKMP 752 IKE_AUTH MID=01 Responder Response
010. .... = Flags: 0x2, Don't fragment
...0 0000 0000 0000 = Fragment Offset: 0
Time to Live: 64
Protocol: UDP (17)
Header Checksum: 0x9f67 [validation disabled]
[Header checksum status: Unverified]
Source Address: 192.168.100.2
Destination Address: 192.168.100.1
[Stream index: 1]
User Datagram Protocol, Src Port: 4500, Dst Port: 4500
UDP Encapsulation of IPsec Packets
Internet Security Association and Key Management Protocol
Initiator SPI: 9b9fe93f827278be
Responder SPI: 521f2836634a0f70
Next payload: Encrypted and Authenticated (46)
Version: 2.0
Exchange type: IKE_AUTH (35)
Flags: 0x08 (Initiator, No higher version, Request)
Message ID: 0x00000001
Length: 768
Payload: Encrypted and Authenticated (46)
0030 9b 9f e9 3f 82 72 78 be 52 1f 28 36 63 4a 0f 70
0040 2e 20 23 08 00 00 00 01 00 00 03 00 23 00 02 e4
0050 40 66 1b c5 e6 1a 8c 94 c5 f3 02 71 23 57 7a 7f
0060 a2 ff 66 33 33 55 20 f1 f5 3f b3 9d 37 17 0c 61
0070 44 93 f2 40 e5 71 50 a2 9d 34 16 42 8e 73 b3 53
0080 54 e2 87 88 17 eb a8 1c 83 16 5b 89 49 e0 4d 8c
0090 f6 55 dc 44 dc b8 6c 79 50 55 50 f1 65 b8 79 a0
00a0 bf e9 f6 1e df 4e e5 3e ce e1 4b 7c ed bc fc ed
00b0 cd 15 e1 c3 4d a9 af 71 1f 91 3b 54 e1 04 27 24
00c0 47 e2 fe 69 58 a3 7d b2 02 e9 cb ce 95 e3 4f e5
00d0 64 c9 50 6d 1e c5 6a d6 42 8c 82 86 12 21 4c 5b
00e0 50 52 be fc 27 27 4e ff 55 1a 6e dd ea df e0 d4
00f0 e2 d7 d8 29 c5 26 53 8d a3 c8 0c 05 8e a0 df 23
0100 cf 68 ca 85 d8 2f 37 35 58 00 ab 2d fc 07 b1 f4
0110 cb f5 96 34 c0 0c ff eb 80 e3 43 b4 48 d6 7c 14
0120 b9 dc 24 4a 27 f9 c8 15 f6 ce fd c7 1d 57 0d 1f
0130 95 29 99 89 36 7e d7 f2 fb 8c 15 42 ed 4b d0 f2
0140 76 0e ee e2 b2 f6 f8 e2 27 25 3a cb f3 d8 d3 c6
0150 0e c0 bd be ea 1d 22 78 2c f6 19 2d 68 11 5e 0d
0160 b1 28 83 f8 30 b3 3b c9 43 a8 b4 58 48 20 83 ba
0170 03 ac a0 bd c1 21 78 ec 5d 34 51 92 9f ba 28 bd
0180 b8 e0 0e 6a 13 60 ce 41 8c 7e 38 23 39 99 82 0e
0190 56 0b 41 04 a0 d7 b4 12 28 00 5b 8b 43 f1 4b f9
01a0 ee 1b 8b 8f 49 43 89 7f a1 b2 91 06 f4 69 5b 9a